دانشگاه صنعتی امیرکبیر
(پلی‌تکنیک تهران)

هوش مصنوعی

پروژه اول

پرند خلیلی ۴۰۱۳۱۰۱۲

پاییز ۱۴۰۴

فهرست مطالب

۱- فصل اول..... ۳

۱-۱ مقدمه..... ۳

۱-۲ ساختار داده‌ها و پیاده‌سازی ۳

۱-۲-۱ ساختار داده‌ای برای نمایش وضعیت‌ها ۳

۱-۲-۲ تابع تولید جانشین‌ها (Successors) ۳

۱-۲-۳ هزینه‌ها و مدیریت بن‌بست‌ها ۳

۱-۲-۴ استفاده از صف و صف اولویت‌دار ٤

۱-۳ الگوریتم‌های جستجو..... ٤

۱-۳-۱ جستجوی عمقی(DFS) ٤

۱-۳-۲ جستجوی گسترده(BFS) ٤

۱-۳-۳ جستجوی با هزینه یکنواخت(UCS) ٤

۱-۳-۴ جستجوی حریصانه(Greedy) ٤

۱-۳-۵ الگوریتم A* ٥

۱-۴ تابع هیوریستیک..... ٥

۲-فصل دوم..... ۷

۲-۱نتایج آزمایش‌ها..... ۷

۲-۲تحلیل نتایج..... ۱۲

۳-فصل سوم..... ۱٤

۳-۱جمع بندی..... ۱٤

۳-۲تحلیل رفتار غیرمنتظره..... ۱٤

۳-۳نتیجه‌گیری نهایی..... ۱٤

۱- فصل اول

۱-۱ مقدمه

در این پروژه، هدف پیاده‌سازی الگوریتم‌های جستجو برای حل بازی **Sokoban** با استفاده از الگوریتم‌های مختلف جستجو از جمله **جستجوی عمقی(DFS)** ، **جستجوی گسترده(BFS)** ، **جستجوی با هزینه یکنواخت (UCS)**، **جستجوی حریصانه (Greedy)** و ***A** است. بازی Sokoban یک بازی پازل است که در آن بازیکن باید جعبه‌ها را به اهداف مشخص شده جابه‌جا کند. هدف این پروژه، طراحی الگوریتم‌های جستجو برای حل این بازی با توجه به موانع و هزینه‌ها بود. علاوه بر این، برای تحلیل رفتار الگوریتم‌ها در مواجهه با مسائل مختلف بازی، از **هیوریستیک**‌های مختلف و **مدیریت بن‌بست‌ها** استفاده شده است.

۱-۲ ساختار داده‌ها و پیاده‌سازی

۱-۲-۱ ساختار داده‌ای برای نمایش وضعیت‌ها

برای مدل‌سازی وضعیت‌های مختلف بازی از یک ساختار State استفاده کرده‌ایم که شامل موقعیت بازیکن و موقعیت جعبه‌ها به صورت frozenset است تا اطمینان حاصل کنیم که وضعیت‌ها تغییرناپذیر هستند. این ساختار داده‌ای به ما این امکان را می‌دهد که وضعیت‌های مختلف را مقایسه کرده و از آن‌ها برای جستجوی مسیر استفاده کنیم.

۱-۲-۲ تابع تولید جانشین‌ها (Successors)

تابع successors در این پروژه وظیفه تولید وضعیت‌های جدید از وضعیت فعلی را بر عهده دارد. این تابع برای هر حرکت ممکن از قبیل حرکت بازیکن و جابه‌جایی جعبه‌ها، وضعیت جدیدی تولید می‌کند. هزینه هر حرکت بسته به نوع آن متفاوت است. به طور کلی، حرکت بازیکن در فضای خالی هزینه کمتری دارد (۱ واحد) و جابه‌جایی جعبه‌ها هزینه بیشتری دارد (۵ واحد).

۱-۲-۳ هزینه‌ها و مدیریت بن‌بست‌ها

در بازی Sokoban ، جابه‌جایی جعبه‌ها در برخی مناطق ممکن است باعث بن‌بست‌های غیرقابل حل شود. در پیاده‌سازی خود از تابع is_simple_corner_deadlock استفاده کرده‌ایم که بررسی می‌کند آیا جعبه در یک گوشه بن‌بست قرار دارد یا خیر. اگر جعبه در یک گوشه بن‌بست قرار گیرد، حرکت آن غیرممکن است و الگوریتم آن را نادیده می‌گیرد.

۱-۲-۴ استفاده از صف و صف اولویت‌دار

برای پیاده‌سازی الگوریتم‌ها از ساختار داده‌های مختلفی استفاده کردیم:

- برای **DFS** از **پشته** به عنوان ساختار داده‌ای استفاده کردیم.
- برای **BFS** از **صف** و برای **UCS**، **Greedy**و **A*** از **صف اولویت‌دار** استفاده کردیم. صف اولویت‌دار در این الگوریتم‌ها برای انتخاب وضعیت با کمترین هزینه (در UCS) یا هیوریستیک بهتر (در Greedy و*A) به کار می‌رود.

۱-۳ الگوریتم‌های جستجو

۱-۳-۱ جستجوی عمقی(DFS)

الگوریتم DFS از **پشته** برای ذخیره‌سازی وضعیت‌های بازدید شده استفاده می‌کند و به صورت عمقی به جستجوی بازی می‌پردازد. این الگوریتم به دلیل ویژگی‌های خود، ممکن است به بن‌بست‌های عمیق برخورد کند و در مواجهه با مسائل بزرگ‌تر عملکرد مناسبی نداشته باشد.

۱-۳-۲ جستجوی گسترده(BFS)

الگوریتم BFS از **صف** برای ذخیره‌سازی وضعیت‌ها استفاده می‌کند و به جستجوی همه‌جانبه می‌پردازد. این الگوریتم گره‌ها را به صورت لایه‌به‌لایه بررسی می‌کند و به دلیل این ویژگی‌اش، همیشه کوتاه‌ترین مسیر را پیدا می‌کند، اما می‌تواند حافظه زیادی مصرف کند.

۱-۳-۳ جستجوی با هزینه یکنواخت(UCS)

درUCS ، از **صف اولویت‌دار** استفاده می‌کنیم تا وضعیت‌هایی که کمترین هزینه را دارند اولویت بیشتری برای جستجو داشته باشند. این الگوریتم گره‌ها را بر اساس هزینه مجموع مسیر (از ابتدای بازی) گسترش می‌دهد و همواره بهترین مسیر را پیدا می‌کند. مشکل اصلی این الگوریتم، زمان و حافظه بالای مصرفی در مسائل با فضای جستجوی بزرگ است.

۱-۳-۴ جستجوی حریصانه(Greedy)

الگوریتم Greedy از **هیوریستیک** برای هدایت جستجو به سمت اهداف استفاده می‌کند. در این الگوریتم، جستجو بر اساس نزدیک‌ترین هدف (با استفاده از فاصله Manhattan) هدایت می‌شود و هیچ توجهی به هزینه‌های واقعی نمی‌کند. این الگوریتم ممکن است سریع باشد اما تضمینی برای یافتن بهترین راه حل ندارد.

۱-۳-۵ الگوریتم *A

الگوریتم *A از **هیوریستیک** و **هزینه واقعی** برای هدایت جستجو استفاده می‌کند. این الگوریتم هزینه‌های مسیر تا کنونی (g) و تخمین فاصله تا هدف (h) را با هم ترکیب کرده و گره‌ها را بر اساس کمترین مجموع آن‌ها اولویت‌بندی می‌کند *A .از الگوریتم‌های بسیار کارآمد است و به دلیل ترکیب جستجو با هزینه واقعی و تخمین هدف، می‌تواند مسیر بهینه را سریع‌تر پیدا کند.

۱-۴ تابع هیوریستیک

در این پروژه برای الگوریتم‌های **Greedy** و ***A** از هیوریستیکی استفاده شده که ترکیبی از **فاصله Manhattan** و **هزینه‌های اضافی** برای مدیریت بن‌بست‌ها و جعبه‌های غیرقابل حرکت است. این هیوریستیک به طور خاص ویژگی‌های زیر را دارد:

۱. **فاصله Manhattan بین بازیکن و اهداف:** این معیار برای تخمین فاصله بازیکن از نزدیک‌ترین هدف به کار می‌رود و به الگوریتم کمک می‌کند تا جستجو را به سمت هدف نزدیک‌تر هدایت کند. این ویژگی به دلیل سادگی و کارایی آن در مسائل مشابه به خوبی جواب می‌دهد و الگوریتم را به صورت مؤثری هدایت می‌کند.

۲. **فاصله Manhattan بین جعبه‌ها و اهداف:** این محاسبه کمک می‌کند تا جعبه‌ها به اهداف نزدیک‌تر شوند و مسیرهای بهینه برای جابه‌جایی جعبه‌ها پیدا شود. این ویژگی مهم است زیرا جابه‌جایی جعبه‌ها بخش عمده‌ای از هزینه‌ها را تشکیل می‌دهد و هدایت الگوریتم بر اساس آن باعث می‌شود که جعبه‌ها سریع‌تر به اهداف برسند.

۳. **هزینه بن‌بست‌ها و جعبه‌های غیرقابل حرکت:** جعبه‌هایی که در بن‌بست‌ها قرار دارند، هزینه اضافی دریافت می‌کنند. این هزینه‌ها باعث می‌شود الگوریتم از مسیرهایی که جعبه‌ها به بن‌بست می‌رسند، اجتناب کند. این ویژگی به جلوگیری از گسترش مسیرهای بی‌ثمر و شبیه‌سازی جستجو در شرایط واقعی بازی کمک می‌کند.

٤. **هزینه جعبه‌هایی که هنوز به هدف نرسیده‌اند:** جعبه‌هایی که به اهداف نرسیده‌اند، هزینه اضافی دریافت می‌کنند تا الگوریتم را مجبور به حرکت دادن این جعبه‌ها به سمت هدف کند. این ویژگی به الگوریتم کمک می‌کند تا جعبه‌ها را به سرعت به اهداف هدایت کند و مسیرهای طولانی‌تر را در اولویت قرار ندهد.

ویژگی‌های هیوریستیک و تاثیر آن در الگوریتم‌ها

- **هیوریستیک غیرخوش‌بینانه** :این هیوریستیک خوش‌بینانه نیست و به دلیل افزودن هزینه‌های اضافی برای بن‌بست‌ها و جعبه‌های غیرقابل حرکت، ممکن است گاهی مسیر بهینه را پیدا نکند. با این حال، این ویژگی باعث می‌شود که الگوریتم از مسیرهای بی‌ثمر و بن‌بست‌ها اجتناب کند.

- **هزینه‌های اضافی برای جعبه‌ها و بن‌بست‌ها** :این هزینه‌ها موجب می‌شوند که الگوریتم به سمت جعبه‌هایی که احتمال بن‌بست دارند نرود و از جابه‌جایی‌های غیرممکن جلوگیری کند.

در نتیجه، این هیوریستیک به طور مؤثری جستجو را به سمت اهداف هدایت می‌کند و از مسیرهای بی‌ثمر و بن‌بست‌ها جلوگیری می‌کند، در حالی که در برخی موارد ممکن است به دلیل غیرخوش‌بینانه بودن، هزینه‌ها را بیشتر از آنچه که واقعاً هستند تخمین بزند.

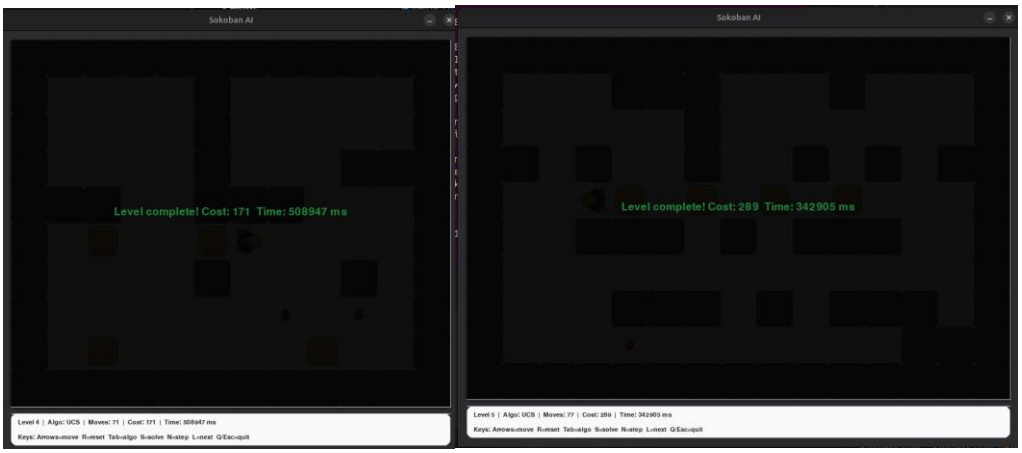
```python
def heuristic_function(state: State, problem: SokobanProblem) -> int:
    # Manhattan distance for each box to the closest goal
    goal_distances = [abs(goal[0] - state.player[0]) + abs(goal[1] - state.player[1]) for goal in problem.goals]
    min_goal_distance = min(goal_distances) if goal_distances else 0

    player_box_distances = [abs(state.player[0] - box[0]) + abs(state.player[1] - box[1]) for box in state.boxes]
    min_player_box_distance = min(player_box_distances) if player_box_distances else 0


    box_distances = []
    for box in state.boxes:
        box_goal_distances = [abs(goal[0] - box[0]) + abs(goal[1] - box[1]) for goal in problem.goals]
        min_box_distance = min(box_goal_distances) if box_goal_distances else 0
        box_distances.append(min_box_distance)

    boxes_on_goal = sum(1 for box in state.boxes if box in problem.goals)
    moveable_box_cost = 0
    for box in state.boxes:
        if problem._is_simple_corner_deadlock(box):
            moveable_box_cost += 100

    deadlock_cost = 0
    for box in state.boxes:
        if problem._is_simple_corner_deadlock(box):
            deadlock_cost += 200

    subgoal_cost = 0
    if boxes_on_goal < len(state.boxes):
        subgoal_cost = 7 * (len(state.boxes) - boxes_on_goal)

    heuristic_value = min_goal_distance + sum(box_distances) + deadlock_cost + moveable_box_cost + subgoal_cost + min_player_box_d
    return heuristic_value
```

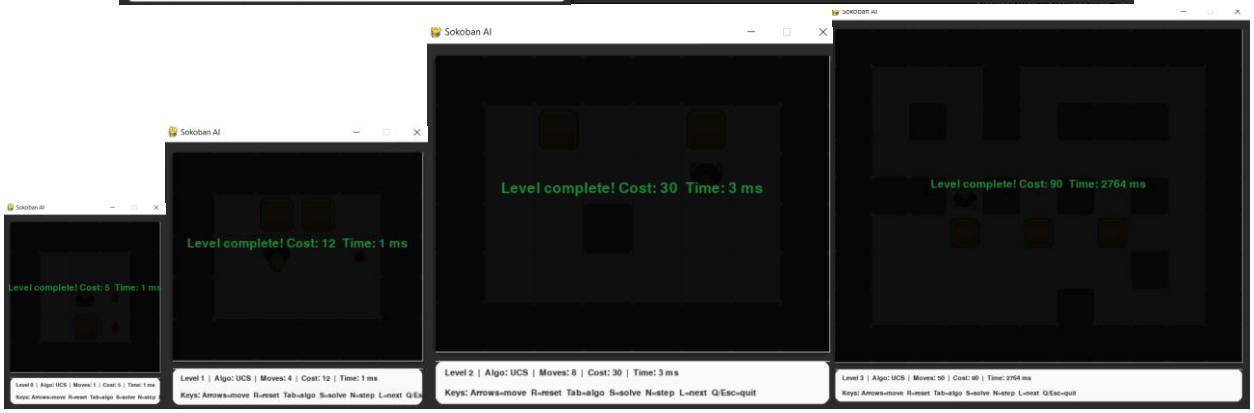
۲- فصل دوم

۲-۱ نتایج آزمایش‌ها

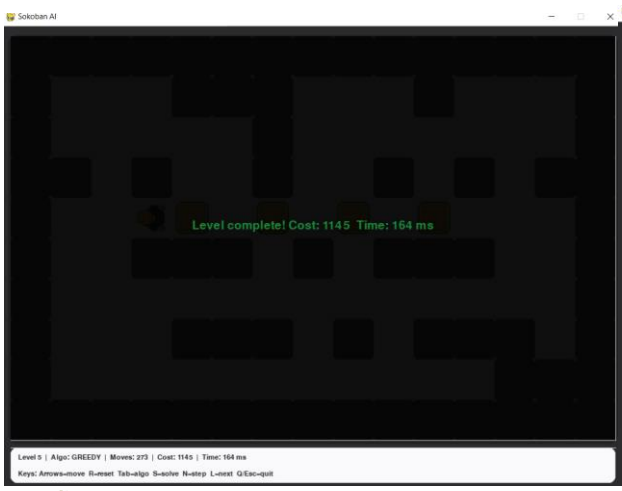
تمامی الگوریتم‌ها بر روی لول‌های مختلف بازی آزمایش شدند. نتایج نشان داد که:

- UCS : این الگوریتم زمان بیشتری برای حل مسائل بزرگ‌تر صرف می‌کند، زیرا تمام وضعیت‌ها را با هزینه واقعی بررسی می‌کند. به‌ویژه در لول‌های بزرگ‌تر که تعداد وضعیت‌ها زیاد است، مصرف حافظه و زمان به طور چشم‌گیری افزایش می‌یابد. به عنوان مثال، در لول‌های با پیچیدگی بیشتر، UCS مجبور به بررسی تمام مسیرها می‌شود، که منجر به زمان اجرا و مصرف حافظه بالا می‌شود.

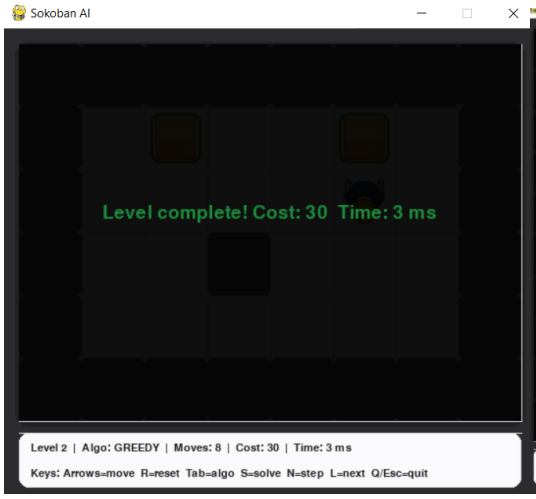




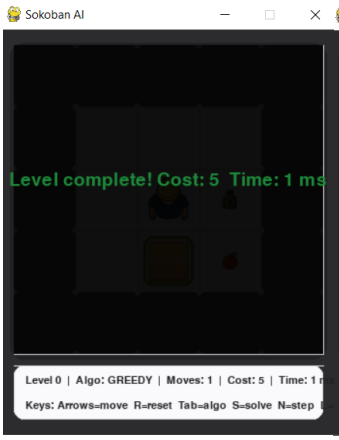
- Greedy: الگوریتم Greedy سریع‌تر عمل می‌کند، زیرا از هیوریستیک برای هدایت جستجو به سمت هدف استفاده می‌کند. این الگوریتم معمولاً سریع‌تر از UCS عمل می‌کند، اما ممکن است مسیری بهینه پیدا نکند. در برخی از آزمایش‌ها، مشاهده شد که این الگوریتم در مسیرهایی قرار گرفت که به بن‌بست رسید یا جعبه‌ها را به سم‌ها جابه‌جا کرد، زیرا به هزینه‌های واقعی توجهی نمی‌کند و تنها به نزدیک‌ترین هدف از نظر هیوریستیک نگاه می‌کند.

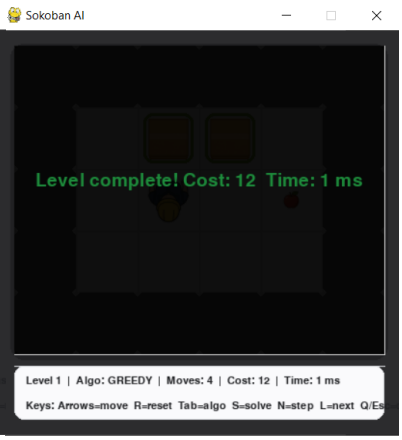

Sokoban AI
Level complete! Cost: 1145 Time: 164 ms
Level 5 | Algo: GREEDY | Moves: 273 | Cost: 1145 | Time: 164 ms
Keys: Arrows=move R=reset Tab=algo S=solve N=step L=next Q/Esc=quit

Sokoban AI
Level complete! Cost: 214 Time: 46 ms
Level 3 | Algo: GREEDY | Moves: 126 | Cost: 214 | Time: 46 ms
Keys: Arrows=move R=reset Tab=algo S=solve N=step L=next Q/Esc=quit

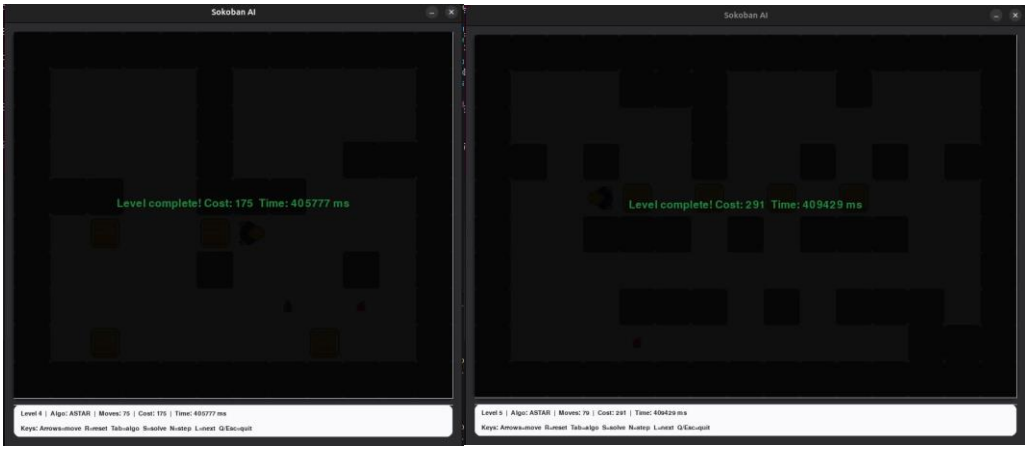

Sokoban AI
Level complete! Cost: 30 Time: 3 ms
Level 2 | Algo: GREEDY | Moves: 8 | Cost: 30 | Time: 3 ms
Keys: Arrows=move R=reset Tab=algo S=solve N=step L=next Q/Esc=quit

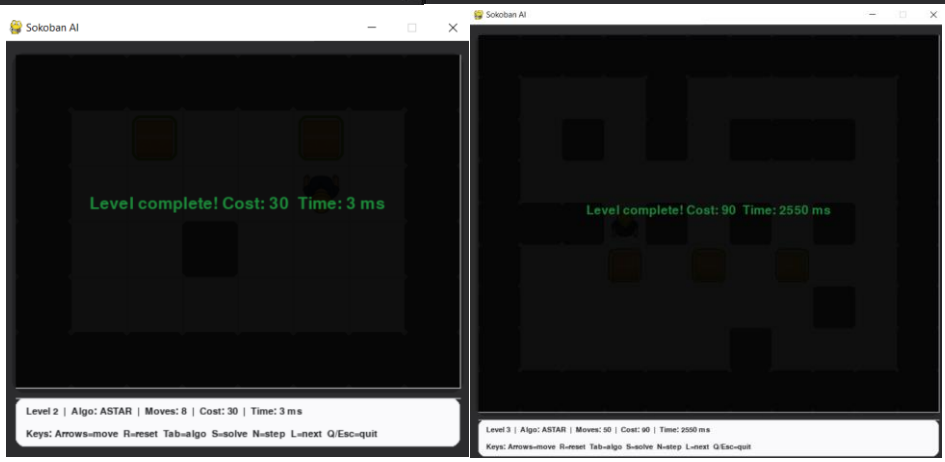
Level complete! Cost: 431 Time: 54 ms
Level 4 | Algo: GREEDY | Moves: 130 | Cost: 431 | Time: 54 ms
Keys: Arrows=move R=reset Tab=algo S=solve N=step L=next Q/Esc=quit

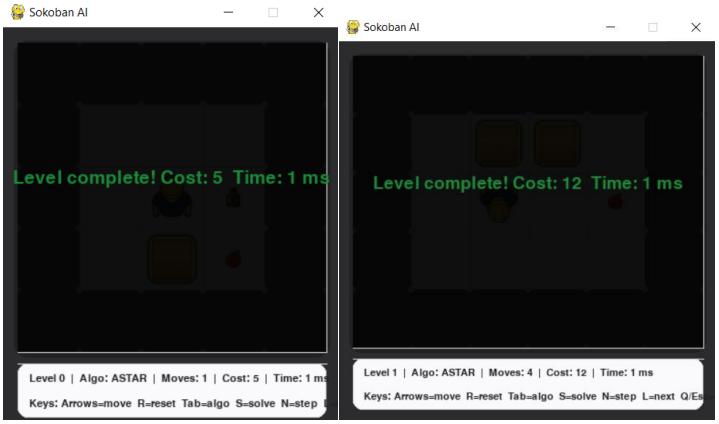

Sokoban AI
Level complete! Cost: 5 Time: 1 ms
Level 0 | Algo: GREEDY | Moves: 1 | Cost: 5 | Time: 1 m
Keys: Arrows=move R=reset Tab=algo S=solve N=step


Sokoban AI
Level complete! Cost: 12 Time: 1 ms
Level 1 | Algo: GREEDY | Moves: 4 | Cost: 12 | Time: 1 ms
Keys: Arrows=move R=reset Tab=algo S=solve N=step L=next Q/Es

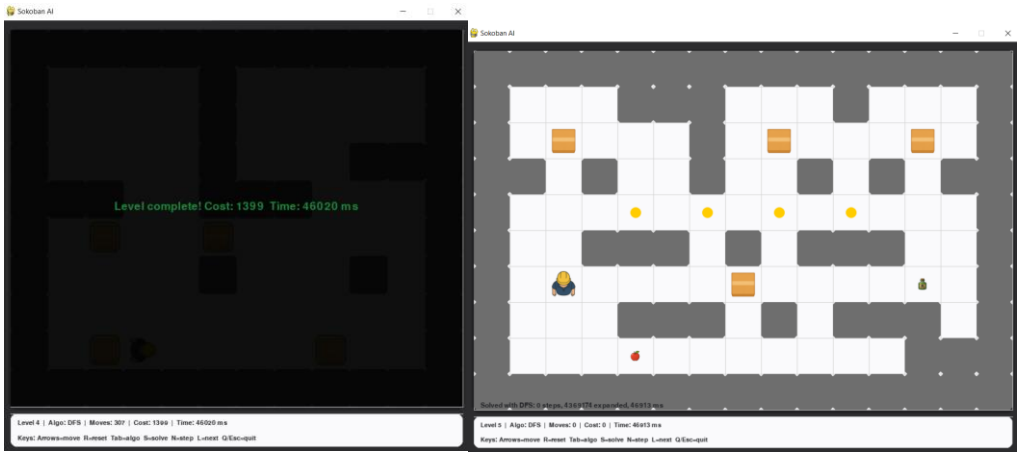
- *A : الگوریتم *A به عنوان الگوریتمی متعادل، در بیشتر مسائل بهترین عملکرد را داشت. این الگوریتم از ترکیب هزینه واقعی و هیوریستیک برای هدایت جستجو استفاده می‌کند و در آزمایش‌ها توانست در اکثر مواقع بهترین مسیر را با کمترین زمان اجرا پیدا کند.

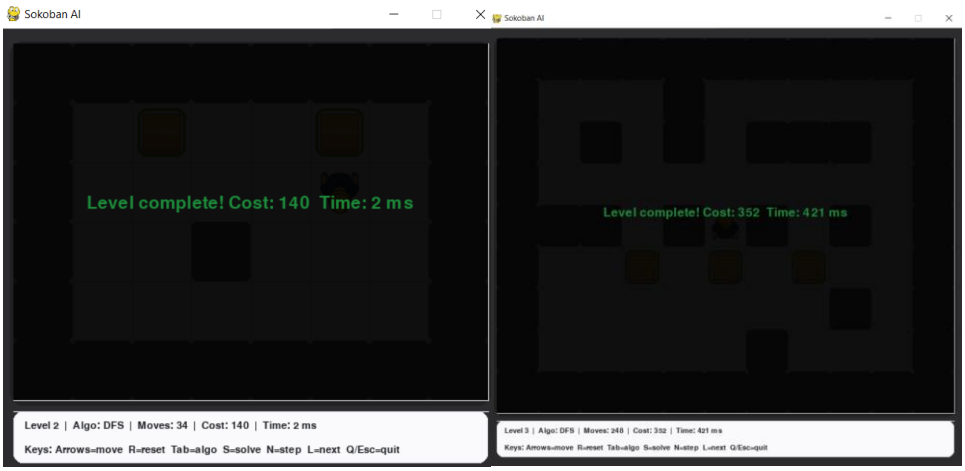


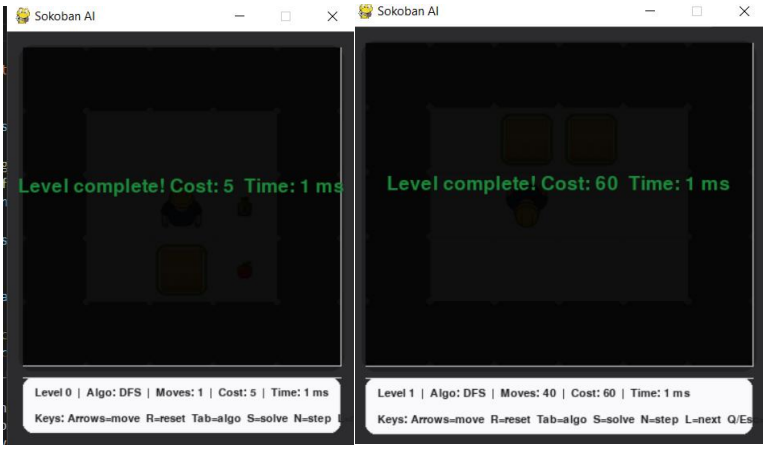




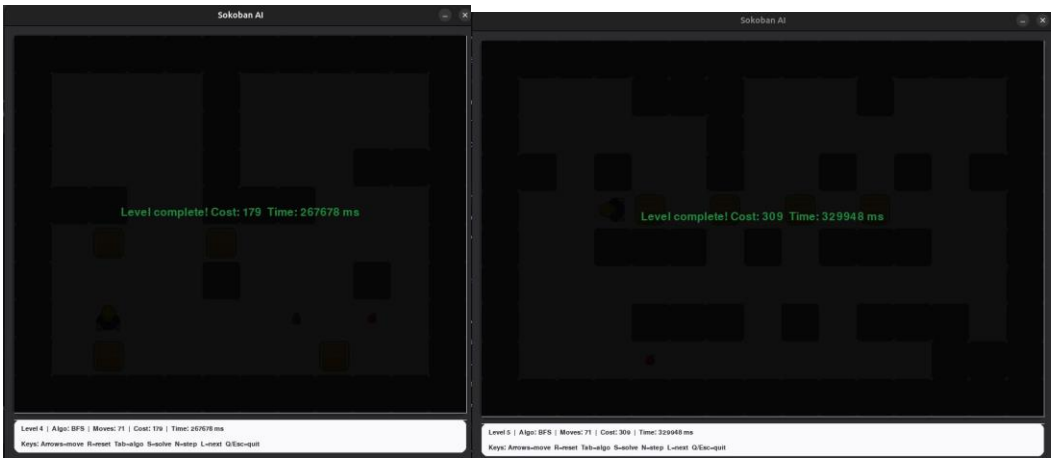
- DFS : جستجوی عمقی (DFS) به طور قابل توجهی عملکرد ضعیف‌تری داشت. این الگوریتم به دلیل جستجو در عمق زیاد و عدم توجه به هزینه‌ها، ممکن است در مواجهه با مشکلات پیچیده یا لول‌های بزرگ‌تر با کمبود حافظه مواجه شود. به دلیل اینکه در DFS به گره‌های زیادی پرداخته می‌شود، در برخی از آزمایش‌ها(مسیر آخر) حتی به علت محدودیت عمق (با استفاده از مقدار max_depth برابر با ۳۰۰) نتواستیم جواب مناسبی پیدا کنیم.

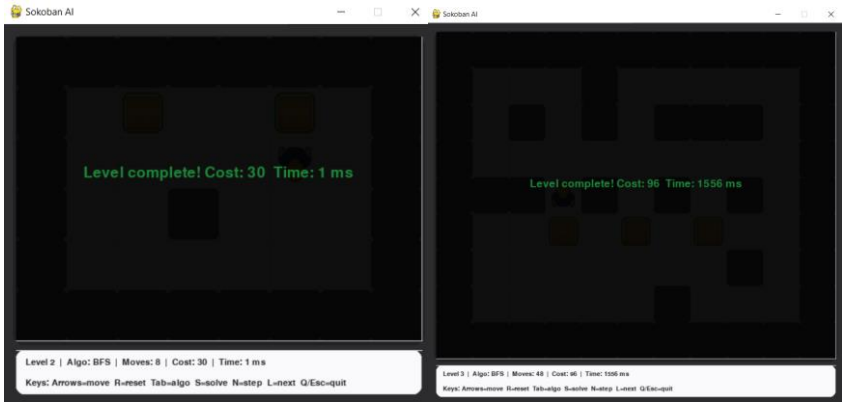


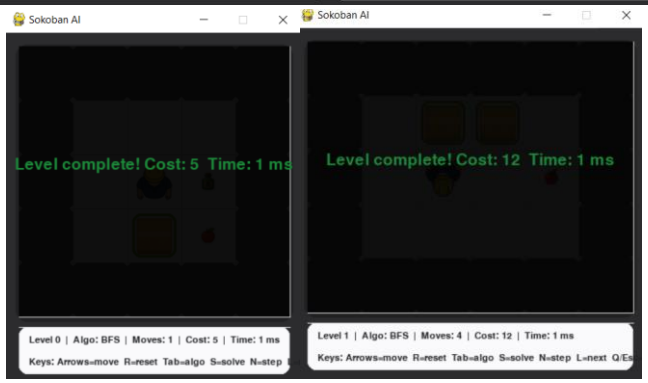
- 



- BFS :الگوریتم جستجوی گسترده (BFS) عملکرد خوبی در یافتن پاسخ بهینه دارد، زیرا گره‌ها را به صورت لایه‌به‌لایه بررسی می‌کند. با این حال، BFS به دلیل استفاده از صف برای ذخیره‌سازی وضعیت‌ها، حافظه زیادی مصرف می‌کند. در لول‌های بزرگ‌تر، این مصرف حافظه می‌تواند بسیار بالا باشد و خیلی طول بکشد به همین علت.



- 



۲-۲ تحلیل نتایج

در لول‌های بزرگ‌تر، زمان اجرای الگوریتم‌ها به دلیل افزایش تعداد گره‌های گسترش‌یافته بیشتر می‌شود. این موضوع به علت گسترش بیشتر فضای جستجو است. در آزمایش‌هایی که در لول‌های پیچیده‌تر انجام شد، **UCS** به دلیل بررسی تمام مسیرها و گره‌ها، مصرف حافظه و زمان بسیار بیشتری داشت. به طور خاص، در لول‌هایی که تعداد وضعیت‌ها و جعبه‌ها زیاد بود، UCSبه شدت کند عمل کرد و مصرف حافظه زیادی داشت.

در مقایسه، **Greedy** به دلیل توجه صرف به هیوریستیک و بی‌توجهی به هزینه‌های واقعی، سریع‌تر از UCS عمل کرد. با این حال، چون فقط به نزدیک‌ترین هدف نگاه می‌کند و از هزینه‌های واقعی غافل است، ممکن است مسیرهای بهینه را پیدا نکند و به بن‌بست برسد.

A* به عنوان یک الگوریتم متعادل، در اکثر موارد بهترین عملکرد را داشت. این الگوریتم قادر بود مسیر بهینه را پیدا کند و به دلیل ترکیب هزینه واقعی و هیوریستیک، در زمان کمتری از سایر الگوریتم‌ها به جواب رسید.

DFS به دلیل محدودیت عمق و همچنین عدم توجه به هزینه‌ها، در حل مسائل بزرگ‌تر مشکل داشت. به خصوص در لول‌های بزرگتر و پیچیده‌تر که تعداد گره‌ها زیاد است، این الگوریتم با کمبود حافظه مواجه می‌شود و به دلیل جستجوی عمیق در مسیرها، ممکن است جواب را پیدا نکند.

- **تحلیل رفتار غیرمنتظره**

در برخی از آزمایش‌ها مشاهده شد که در الگوریتم‌های مختلف، به خصوص **Greedy** و **A***، حتی در مواقعی که هزینه‌ها باید مدنظر قرار می‌گرفتند، جعبه‌ها به **سم‌ها** جابه‌جا شدند یا بازیکن به سم‌ها برخورد کرد. این رفتار غیرمنتظره در چندین الگوریتم مشاهده شد که توضیح آن به شرح زیر است:

۱. **الگوریتمGreedy**

- الگوریتم **Greedy** فقط به **هیوریستیک** (فاصله Manhattan به نزدیک‌ترین هدف) توجه می‌کند و **هزینه واقعی** را در نظر نمی‌گیرد. در نتیجه، این الگوریتم ممکن است به مسیری حرکت کند که کوتاه‌ترین مسیر به نظر برسد اما در این مسیر **سم** قرار دارد. این الگوریتم برای رسیدن به هدف، صرفاً به نزدیک‌ترین هدف از نظر هیوریستیک توجه می‌کند، حتی اگر این مسیر شامل حرکت به سم‌ها باشد.
- در برخی لول‌ها مشاهده شد که گرچه مسیر مستقیم به هدف کوتاه‌تر بود، اما به دلیل وجود سم‌ها در آن مسیر، الگوریتم تصمیم می‌گرفت مسیری دیگر را امتحان کند که ممکن بود هزینه بیشتری به همراه داشته باشد.

۲. **الگوریتم A***

- در **A*** نیز که از ترکیب **هیوریستیک** و **هزینه واقعی** برای یافتن مسیر بهینه استفاده می‌کند، ممکن بود باز هم مسیرهای نزدیک‌تر به هدف انتخاب شوند، حتی اگر این مسیرها شامل سم‌ها بودند. این الگوریتم، در تلاش برای پیدا کردن بهینه‌ترین مسیر، گاهی به دلیل اولویت دادن به نزدیک‌ترین هدف ممکن بود مسیرهایی را انتخاب کند که در آنها سم‌ها قرار داشتند. به طور مشابه، در این الگوریتم نیز وقتی مسیر کوتاه‌تر به هدف در نظر گرفته می‌شود، **سم‌ها** ممکن است بخشی از مسیر باشند و الگوریتم به آنها توجه نکند.

۳. **الگوریتم‌های DFS و BFS**

- در **DFS** و **BFS**، هدف اصلی فقط رسیدن به هدف است و الگوریتم‌ها توجهی به هزینه واقعی یا هیوریستیک ندارند. بنابراین، مسیرهایی که به هدف نزدیک‌تر هستند، حتی اگر سم‌ها را شامل شوند، انتخاب می‌شوند. این رفتار در هر دو الگوریتم مشهود است، زیرا این الگوریتم‌ها فقط به **رسیدن به هدف** توجه دارند و ممکن است به مسیرهایی که از سم‌ها عبور می‌کنند، وارد شوند.
- در این الگوریتم‌ها به دلیل عدم توجه به هزینه‌های واقعی و نبود هیوریستیک، جعبه‌ها ممکن است به سم‌ها جابه‌جا شوند یا بازیکن با آنها برخورد کند.

۳-فصل سوم

۳-۱ جمع بندی

در این پروژه، الگوریتم‌های مختلف جستجو برای حل بازی **Sokoban** پیاده‌سازی و آزمایش شدند. نتایج نشان داد که هر الگوریتم ویژگی‌ها و چالش‌های خاص خود را دارد که بسته به نوع مسئله، می‌تواند مناسب یا غیرمناسب باشد. در میان الگوریتم‌ها، **A*** به عنوان الگوریتمی متعادل، بهترین عملکرد را از نظر زمان اجرا و یافتن مسیر بهینه داشت. این الگوریتم با استفاده از ترکیب هزینه واقعی و هیوریستیک، توانست در اکثر مواقع به جواب‌های بهینه در کمترین زمان برسد. در حالی که **Greedy** به دلیل بی‌توجهی به هزینه‌های واقعی و تمرکز صرف بر هیوریستیک، سریع‌تر عمل کرد اما ممکن بود به مسیری ناپایدار یا به بن‌بست‌ها برسد.

از سوی دیگر، **UCS** به دلیل بررسی تمامی مسیرها، به ویژه در مسائل بزرگ‌تر، زمان و حافظه زیادی مصرف کرد و در لول‌های پیچیده‌تر سرعت بسیار کندی داشت. **DFS** و **BFS** نیز در برخی از آزمایش‌ها به دلیل توجه نداشتن به هزینه‌ها و یا محدودیت‌های حافظه‌ای با مشکلاتی مواجه شدند. در DFS به دلیل جستجو در عمق زیاد، و در BFS به دلیل مصرف بالای حافظه، این الگوریتم‌ها نتوانستند در برخی از لول‌ها به پاسخ مناسبی برسند.

۳-۲ تحلیل رفتار غیرمنتظره

در برخی از آزمایش‌ها، مشاهده شد که الگوریتم‌ها به دلیل عدم توجه کامل به هزینه‌های واقعی، ممکن بود جعبه‌ها را به سمت **سم‌ها** جابه‌جا کنند یا بازیکن به سم‌ها برخورد کند. این امر به دلیل تمرکز بیش از حد بر **نزدیک‌ترین هدف** و عدم توجه کافی به **هزینه‌های واقعی** حرکت بود. در این شرایط، الگوریتم‌ها برای رسیدن به هدف، مسیرهایی را انتخاب کردند که کوتاه‌ترین مسیر به نظر می‌رسید اما در این مسیرها سم‌ها یا موانع دیگر وجود داشت. این موضوع در **Greedy** مشهودتر بود، چرا که فقط به هیوریستیک توجه می‌کند و در **A*** نیز مشاهده شد که گاهی به دلیل اولویت دادن به نزدیک‌ترین هدف، سم‌ها بخشی از مسیر می‌شدند.

در **DFS** و **BFS** نیز که هدف اصلی فقط رسیدن به هدف بود، بدون توجه به هزینه‌ها یا هیوریستیک، مسیرهایی که به هدف نزدیک‌تر بودند حتی اگر شامل سم‌ها می‌شدند، انتخاب شدند. این رفتار در این الگوریتم‌ها نشان می‌دهد که بدون در نظر گرفتن هزینه‌های واقعی و هیوریستیک مناسب، احتمال برخورد با سم‌ها بیشتر می‌شود.

۳-۳ نتیجه‌گیری نهایی

این پروژه نشان داد که انتخاب الگوریتم مناسب بسته به ویژگی‌های مسئله می‌تواند تأثیر زیادی بر کارایی و دقت حل مسئله داشته باشد**A*** به دلیل ترکیب مناسب هزینه‌ها و هیوریستیک، بهترین عملکرد را در اکثر آزمایش‌ها داشت . **Greedy** سریع‌ترین بود اما به دلیل غفلت از هزینه‌ها ممکن است مسیر بهینه را پیدا نکند. **UCS** در مسائل پیچیده‌تر به دلیل مصرف زیاد حافظه و زمان، محدودیت‌هایی داشت. بنابراین، برای حل مسائل مشابه، استفاده از

الگوریتم‌هایی مانند **A*** که از ترکیب هزینه‌های واقعی و هیوریستیک استفاده می‌کنند، به نظر می‌رسد که بهترین گزینه باشد.